

**RAPPORT DE STAGE
DE MAGISTÈRE**



**Étude d'un protocole de chiffrement
post-quantique**

Stagiaire

Raphael LEONARDI

Établissement

Université Paris-Saclay

Période

5 mai 2025 - 4 juillet 2025

Encadrant

Christophe Levrat

Introduction

J'ai effectué mon stage de fin d'année à l'INRIA, au sein de l'équipe GRACE, sous la direction du post-doctorant Christophe Levrat. Le but de ce stage était de découvrir et d'implémenter un protocole de chiffrement basé sur les codes correcteurs, nommé Hamming Quasi Cyclid (HQC), en langage C.

1.1 Contexte du stage

Au cours du second semestre de cette année, j'ai choisi d'effectuer un stage TER. Etant intéressé par le domaine de la cryptographie, j'ai contacté Alain Couvreur, le chef d'équipe de l'équipe-projet GRACE, dont le travail de recherche se concentre sur la cryptographie. Un membre de son équipe, Christophe Levrat a répondu positivement à ma demande de

TER, et m'a proposé un sujet traitant de preuves Zero-Knowledge. Je venais donc une fois par semaine dans les locaux de l'équipe GRACE. Je fus très bien accueilli, l'ambiance de travail et d'équipe était très agréable. C'est pour cela que lorsque mon tuteur m'a proposé d'effectuer mon stage de fin d'année au même endroit j'ai accepté. Le 11 mars

2025, l'organisation américaine du NIST a choisi HQC, développé par une équipe de chercheurs principalement français comme cinquième algorithme pour le chiffrement post-quantique, le NIST travaille depuis plus de huit ans sur des algorithmes de chiffrement que même un ordinateur quantique ne peut déchiffrer. L'équipe de HQC a publié sur le site la documentation pour implémenter le protocole de chiffrement de HQC, sur laquelle je base mon code. Vous pouvez consulter la documentation officielle du protocole HQC à l'adresse suivante :

<https://pqc-hqc.org/documentation.html>

Cette annexe se base sur plusieurs livres et cours que l'on peut trouver sur Internet.

1.2 Etat de l'art sur HQC

Avec l'avènement des ordinateurs quantiques, plusieurs problèmes mathématiques fondamentaux utilisés en cryptographie classique deviennent vulnérables. C'est notamment le cas du problème du logarithme discret, sur lequel reposent des systèmes cryptographiques comme Diffie-Hellman ou l'algorithme d'ElGamal. Les algorithmes quantiques, comme celui de Shor, peuvent résoudre ce problème en temps polynomial, rendant ainsi ces

systèmes obsolètes face à une menace quantique réaliste. Cette fragilité a poussé la communauté scientifique à développer des schémas cryptographiques résistants aux attaques quantiques, regroupés sous le terme de cryptographie post-quantique. Parmi les alterna-

tives prometteuses figure la famille des cryptosystèmes à base de codes, dont fait partie HQC (Hamming Quasi-Cyclic). Cet algorithme repose non pas sur le problème du logarithme discret, mais sur la difficulté de décoder un mot bruité dans un code linéaire, un problème réputé difficile même pour un ordinateur quantique. HQC combine des structures quasi-cycliques pour optimiser les performances et la taille des clés, tout en s'appuyant sur le problème de décodage syndrome dans un espace de Hamming, qui reste intraitable malgré les avancées en informatique quantique.

À l'heure actuelle, HQC représente l'un des états de l'art (SOTA) en matière de cryptographie post-quantique. Il fait partie des finalistes du processus de standardisation lancé par le NIST pour sélectionner les futurs standards cryptographiques résistants aux ordinateurs quantiques. Sa robustesse théorique, associée à une mise en œuvre efficace, lui confère un avantage stratégique, notamment pour les systèmes embarqués ou les communications sécurisées à long terme. Contrairement aux systèmes vulnérables basés sur le logarithme discret, HQC anticipe la menace quantique en s'appuyant sur des problèmes combinatoires profondément différents, jugés sûrs même face aux capacités exponentielles des algorithmes quantiques.

1.3 Codes correcteurs

L'algorithme HQC utilise des codes correcteurs pour chiffrer et déchiffrer des messages transmis. Vous pouvez trouver dans l'annexe une introduction aux codes correcteurs utilisés par HQC.

Ce document a été élaboré en s'appuyant sur diverses sources académiques que voici :

1. Introduction to Coding Theory, Alain Couvreur : https://www.lix.polytechnique.fr/~alain.couvreur/doc_ens/lecture_notes.pdf
2. *Error Control Coding: Fundamentals and Applications*, Shu Lin et Daniel J. Costello Jr, 1982
3. *The Theory of Error-Correcting Codes*, F. J. MacWilliams et N. J. A. Sloane, North-Holland, 1977.

Les définitions mathématiques et les propriétés des codes linéaires présentées dans ce document sont issues de sources classiques en théorie de l'information et en algèbre linéaire. Les algorithmes et les paramètres spécifiques à HQC sont directement tirés de la documentation officielle du projet.

Problème traité et solution proposée

Voici le schéma d'encryption de HQC :

- **Setup**(1^λ) : produit les paramètres globaux **param** = $(n, k, \Delta, w, w_r, w_e, \ell)$.
- **KeyGen**(**param**) : échantillonne $\mathbf{h} \xleftarrow{\$} \mathcal{R}$, la matrice génératrice $\mathbf{G} \in \mathbb{F}_2^{k \times n}$ de $\mathcal{C}$, $(\mathbf{x}, \mathbf{y}) \xleftarrow{\$} \mathcal{R}_w \times \mathcal{R}_w$, définit **sk** = $(\mathbf{x}, \mathbf{y})$ et **pk** = $(\mathbf{h}, \mathbf{s} = \mathbf{x} + \mathbf{h} \cdot \mathbf{y})$, et retourne **(pk, sk)**.
- **Encrypt**(**pk**, **m**) : génère $\mathbf{e} \xleftarrow{\$} \mathcal{R}_{w_e}$, $\mathbf{r} = (\mathbf{r}_1, \mathbf{r}_2) \xleftarrow{\$} \mathcal{R}_{w_r} \times \mathcal{R}_{w_r}$, définit $\mathbf{u} = \mathbf{r}_1 + \mathbf{h} \cdot \mathbf{r}_2$ et $\mathbf{v} = \text{tronquer}(\mathbf{m}\mathbf{G} + \mathbf{s} \cdot \mathbf{r}_2 + \mathbf{e}, \ell)$, retourne **c** = $(\mathbf{u}, \mathbf{v})$.
- **Decrypt**(**sk**, **c**) : retourne $\mathcal{C}\text{Decode}(\mathbf{v} - \mathbf{u} \cdot \mathbf{y})$.

La formulation $a \xleftarrow{\$} \mathcal{R}$ signifie que l'on tire au hasard un élément a dans l'ensemble $\mathcal{R}$. L'ensemble $\mathcal{R}_{w_e}$ correspond à l'ensemble des tableaux de bits ayant un poids de Hamming (le nombre de bits égaux à 1) de w_e . On peut diviser l'implémentation de ce schéma en 3 parties :

1. La génération de clés, l'addition et la multiplication de tableaux binaires
2. L'encodage des messages
3. Le décodage des messages

2.1 Implémentation tableaux binaires

Nous allons devoir manipuler des tableaux binaires contenant plus de 30000 bits. Nous proposons la solution de créer un alias en C, et de stocker ces tableaux dans des pointeurs contenant des entiers de 64 bits :

```
typedef uint64_t word;

typedef struct message{
    int size;
    word *words;
} message;
```

Génération aléatoire

Supposons que l'on veuille générer un tableau binaire de longueur n et de poids de Hamming de poids w :

- On génère un tableau de taille n , que l'on nommera **tab** et ou, **tab**[i] = i
- On mélange ce tableau avec l'algorithme de Fisher-Yates
- Ensuite on renvoie un tableau dont les bits égaux à 1 sont les w premières valeurs de **tab**.

Addition, soustraction

Supposons que l'on soit en présence de deux tableaux binaires **tab1** et **tab2**. Ce sont des polynômes à coefficients dans $\mathbb{F}_2$. Les opérations d'addition et de soustraction sont les mêmes et l'on a simplement à effectuer une opération **XOR** entre chaque élément de **tab1** et **tab2**.

Produit

Le produit de **tab1** et **tab2** se fait en 2 étapes. Tout d'abord on fait une multiplication entre les deux polynômes :

Input: Deux vecteurs a et b de polynômes sur 64 bits, de tailles a_len et b_len

Output: Le vecteur $result = a \times b$

Créer un vecteur temporaire tmp de taille $a_len + b_len$;

Initialiser tous les éléments de tmp à 0;

for $i \leftarrow 0$ **to** $a_len - 1$ **do**

for $j \leftarrow 0$ **to** $b_len - 1$ **do**

$high, low \leftarrow \text{multiply_polynomials}(a[i], b[j]);$

$tmp[i + j] \leftarrow tmp[i + j] \oplus low;$

$tmp[i + j + 1] \leftarrow tmp[i + j + 1] \oplus high;$

end

end

Ensuite on doit appliquer le quotient par $X^{17664} - 1$, en appliquant des opération **XOR** entre les coefficients de tmp et les coefficients de $X^{17664} - 1$.

On a implémenté les algorithmes nécessaires aux opérations sur les tableaux binaires nécessaires au chiffrement, maintenant, intéressons nous à l'encodage et au décodage utilisés par HQC.

2.2 Encodage par Reed-Solomon et Reed-Muller

Maintenant que nous avons implémenté les opérations sur les tableaux binaires, nous allons pouvoir implémenter l'encodage des messages.

Le premier encodage que nous allons implémenter est l'encodage de Reed-Solomon. Cet encodage est basé sur des calculs de restes de polynômes. Nous allons donc devoir implémenter ces polynômes. Nous allons utiliser la structure de données suivante :

```
typedef struct {
    int degre; //le degré du polynôme
    int longueur; //la longueur du polynôme (ou le degré max +1) )
    uint8_t coeffs[]; // les coefficients du polynôme
} p256;
```

Les coefficients du polynôme sont des éléments de $\mathbb{F}_{256}$, c'est-à-dire des entiers entre 0 et 255, le type `uint8_t` est donc idéal pour représenter ces coefficients. Dans la documentation de HQC, on peut trouver un tableau des différentes puissances et logarithmes de α , où α est un élément de $\mathbb{F}_{256}$, qui est la racine primitive de l'anneau $\mathbb{F}_{256}$. En se basant sur ce tableau, et les propriétés suivantes :

$$a \times b = \exp(\log(a) + \log(b) \mod 256) \quad a^{-1} = a^{255} \quad a \div b = a \times (b^{-1})$$

$$x^y = x^{(y \mod 256)} \quad a + b = a - b = a \oplus b$$

On peut implémenter les opérations de multiplication, division et addition de polynômes sur $\mathbb{F}_{256}$.

Le plus important est l'algorithme de division de polynômes, qui est utilisé pour l'encodage de Reed-Solomon: Le polynôme Q est un polynôme unitaire ici.

Input: Deux polynômes P et Q de degrés respectifs deg_P et deg_Q

Output: Le reste R de la division de P par Q

Initialiser $R = P$;

while $deg(R) \geq deg(Q)$ **do**

Calculer $d = deg_R - deg_Q$;

Calculer $c = R[deg_R]$;

Mettre à jour $R = R - c \cdot X^d \cdot Q$;

end

return R ;

Après avoir implémenté les opérations sur les polynômes, il reste encore une chose à implémenter : des matrices. Pour cela nous allons créer une structure de données pour représenter les matrices, qui seront utilisées pour l'encodage de Reed-Muller et de Reed-Solomon.

```
typedef struct
{
    int height;          int getValue(matrice *m, int i, int j);
    int width;           void setValue(matrice *m, int i, int j, uint8_t value);
    uint8_t* values;
} matrice;
```

Ce dont on a besoin est de pouvoir faire le calcul vecteur-matrice, par la combinaison linéaire des lignes de la matrice et des coefficients du vecteur. Voici l'algorithme:

Input: Un vecteur v de taille n contenant des octets et une matrice G de hauteur m et de largeur n

Output: Un vecteur $result$ de taille n

Initialiser $result$ de taille n avec des zéros;

Initialiser i à 0;

foreach $b \text{ bit} \in v$ **do**

if $b == 1$ **then**

$result \leftarrow result \oplus i\text{ème ligne de } G;$

end

$i \leftarrow i + 1;$

end

return $result;$

Une fois que nous avons implémenté les opérations sur les polynômes et les matrices, nous pouvons implémenter l'encodage de Reed-Solomon et de Reed-Muller. Ce que contiennent les matrices G de Reed-Solomon et de Reed-Muller est expliqué dans l'annexe. En appliquant d'abord l'encodage de Reed-Solomon, puis l'encodage de Reed-Muller et en dupliquant chaque bit pour l'avoir 3 fois, on obtient un message encodé de taille $46 \times 128 \times 3 = 17664$. Ensuite on applique le schéma de chiffrement de HQC, via l'ajout de bruit, et de calculs à base d'addition et de multiplication de tableaux binaires entre le message encodé et les clés publiques et privées.

2.3 Décodage de Reed-Solomon et Reed-Muller

Le processus pour le décodage de Reed-Solomon est bien trop long pour être détaillé ici, mais il est basé sur la recherche de racines du polynôme minimal et l'utilisation de l'algorithme de Berlekamp-Massey pour reconstruire le message original à partir des coefficients du polynôme. En revanche, le décodage de Reed-Muller est plus simple et rapide à décrire, ce que nous allons faire ici. Comme stipulé dans l'annexe, le décodage

de Reed-Muller est basé sur un produit successif entre matrice et vecteur. On a besoin de stocker 7 matrices de taille 128×128 . Mais chacune des lignes de ces matrices possède exactement 2 valeurs non nulles (soit 1 ou -1). Au lieu de stocker les 128 valeurs de chaque ligne, on va stocker les indices des valeurs non nulles et leurs signes.

```
typedef struct{
    char * positions;
    char * signs;
    int m;
} matHadar;
```

Une matrice de Hadamard est composée uniquement des valeurs -1 , 0 et 1 . Sachant qu'il n'y a que deux coefficients non nuls par ligne (égaux à -1 ou 1), on peut optimiser son stockage en ne mémorisant que les ****positions**** de ces coefficients non nuls ainsi que leur ****signe****.

Par exemple, considérons la matrice suivante :

$$\begin{pmatrix} 1 & 1 & 0 & 0 \\ 1 & -1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & -1 \end{pmatrix}$$

Cette matrice peut être représentée de manière compacte par deux tableaux :

- **positions** : (0, 1, 0, 1, 2, 3, 2, 3)
- **signes** : (1, 1, 1, -1, 1, 1, 1, -1)

Cette représentation est utile pour les calculs et permet une meilleure utilisation de la mémoire. On a besoin de calculer une fois au à l'exécution ces matrices, puis on les stocke dans une variable, par exemple dans une structure de ce format :

```
typedef struct{
    matHadar **matrices; //Un tableau de pointeurs vers les matrices de Hadamard
    int m; // le nombre de matrices
} reedDecoder;
```

Comme stipulé dans l'annexe, on calcule le vecteur F , puis l'on calcule avec les matrices de Hadamard le vecteur $\hat{F}$, qui contient en chaque position i la corrélation entre le mot transmis et l'encodage du mot i .

2.4 Remerciements

Avant de passer à la conclusion je tiens à effectuer quelques remerciements. Je tiens à remercier tout d'abord mon encadrant, Christophe Levrat, pour avoir accepté de m'encadrer pour mon TER et pour mon stage de fin d'année. Il m'a permis de découvrir des mondes qui m'étaient alors inconnus : le monde de la recherche et la cryptographie.

Je remercie également Alain Couvreur, qui est la première personne que j'ai contacté, et qui a parlé de moi à son équipe.

Enfin je remercie l'ensemble de l'équipe GRACE, les doctorants, les post-doctorant et les chercheurs. Ils m'ont fait un accueil chaleureux, et je me suis vite senti à l'aise parmi eux.

2.5 Conclusion et perspectives

Ce stage de 2 mois m'a permis de découvrir le monde de la recherche en laboratoire, et de découvrir la cryptographie, un domaine en pleine expansion. J'ai pu mettre en pratique mes connaissances en C et en algèbre linéaire, et j'ai appris à travailler avec des algorithmes complexes de chiffrement.

Ce stage se plaçait dans la suite de logique du TER que j'avais effectué, au sein de la même équipe, supervisé par le même encadrant. J'ai donc pu approfondir mes connaissances en cryptographie, et découvrir un protocole de chiffrement post-quantique, qui est l'un des plus avancés à l'heure actuelle. J'ai du apprendre une partie des mathématiques qui m'étaient totalement inconnues, à savoir les codes correcteurs.

J'ai pu acquérir au cours de ce stage des compétences en cryptographie, qui me seront utiles pour la suite de mon parcours. En effet, bien que je me dirige vers un Master en Intelligence Artificielle, j'ai pu trouver un stage de TER pour l'année prochaine avec un chercheur de l'INRIA qui m'a proposé un sujet liant l'IA et la cryptographie. Donc pour conclure, je peux dire que ce stage m'a permis de découvrir un domaine qui m'intéressait, de découvrir le monde de la recherche, et de me préparer pour la suite de mon parcours académique.

Annexe sur les codes correcteurs

July 12, 2025

Abstract

1 Introduction

Dans cette annexe, nous allons décrire et expliquer le fonctionnement et l'utilité des codes correcteurs utilisés par l'algorithme HQC.

1.1 Définitions

Soit n un entier positif. Nous disons que $\mathcal{C}$ est un code correcteur d'erreurs linéaire si $\mathcal{C}$ est un sous-espace de $\mathbb{F}_q^n$. L'entier n est appelé la longueur de $\mathcal{C}$. La dimension de $\mathcal{C}$ est :

$$k = \dim_{\mathbb{F}_q} \mathcal{C}$$

Un élément de $\mathcal{C}$ sera appelé un mot de code.

1.2 Décrire un code

Il existe deux façons de décrire un code en utilisant des matrices. Il peut être représenté soit comme l'image d'une matrice, soit comme le noyau d'une autre matrice.

Définition 1 (Matrice génératrice). Soit $\mathcal{C}$ un code linéaire $[n, k, d]$. Une matrice génératrice pour $\mathcal{C}$ est une matrice $G \in \mathfrak{M}_{l,n}(\mathcal{F}_q^l)$ avec $l \geq k$, dont les lignes forment un ensemble générateur pour $\mathcal{C}$. Ainsi :

$$\mathcal{C} = \{mG \mid m \in \mathcal{F}_q^l\}$$

Supposons que l'on veuille transmettre un message par un canal bruité, c'est à dire que l'on sait qu'il y aura des erreurs. L'idée des codes correcteurs est d'encoder ce message, l'envoyer et de corriger les erreurs transmises. L'ensemble des messages encodés par un certain code linéaire était restreint, l'idée est de trouver le plus proche mot du code, et supposer que le message transmis était ce mot du code.

La matrice génératrice G est ce qui permet d'encoder un message en un mot de code. Mais elles n'utilisent pas la même convention d'écriture pour le produit matriciel qu'en France.

Supposons que l'on ait un vecteur v contenant n éléments, et une matrice G contenant n lignes de k éléments chacun. Les valeurs k et n sont totalement indépendants, comme l'on cherche à ajouter

de l'information dans un code correcteur, en général $k > n$. Le mot encodé mG est calculé en effectuant les combinaisons linéaires entre les lignes de G et les coefficients de v . Dans les codes correcteurs que nous utiliserons, l'opération $+$ est en réalité un **XOR** réalisé bit à bit.

Définition 2. $\mathbb{F}_{256}$ est un corps fini (également connu sous le nom de corps de Galois) composé de 256 éléments. Il est représenté comme le corps d'ordre 2^8 , généralement construit à l'aide de polynômes sur $\mathbb{F}_2$ avec un module de degré 8. Chaque élément peut être exprimé comme un nombre binaire de 8 bits.

2 Codes correcteurs utilisés par HQC

Pour pouvoir chiffrer et déchiffrer des messages, l'algorithme HQC utilise deux codes correcteurs concaténés : les codes de Reed-Solomon et de Reed-Muller. Nous allons expliquer dans cette section en quoi consiste ces codes.

2.1 Code de Reed-Solomon

Le premier encodage que nous utilisons est l'encodage Reed-Solomon, ou plus spécifiquement l'encodage Reed-Solomon raccourci. Il possède les paramètres spécifiques suivants :

- Longueur de bloc : n_1
- Dimension : k
- Nombre de symboles de parité $n - k = 2\delta$, avec δ , la capacité de correction du code et k le nombre de bits d'information
- Distance minimale $d_{min} = 2\delta + 1$

Les éléments du code Reed-Solomon sont des coefficients dans $\mathbb{F}_{256}$ que nous traitons comme des polynômes. Soit α un élément primitif dans $\mathbb{F}_{256}$, le polynôme générateur de $RS[n, k, \delta]$ est donné par :

$$g(x) = (x + \alpha)(x + \alpha^2) \dots (x + \alpha^{2\delta})$$

Nous notons par

$$u(x) = u_0 + u_1x + \dots + u_{k-1}x^{k-1}$$

le polynôme correspondant au message

$$u = (u_0, u_1, \dots, u_{k-1})$$

à encoder, et par $g(x)$ le polynôme générateur.

L'encodage de $u(x)$ est $b(x) + x^{n-k}u(x)$, avec $b(x)$ étant le reste de la division polynomiale de $u(x)$ par $g(x)$.

Nous construisons la matrice génératrice en appliquant l'encodage aux éléments canoniques de la base $1, x^1, x^2, \dots, x^{k-1}$.

Pour cela, définissons les polynômes $b_i(x)$, $1 \leq i \leq k$ comme les restes de la division polynomiale de x^i par $g(x)$ et les $b_{i,1}, \dots, b_{i,n-k}$ représentent les coefficients de $b_i(x)$.

Alors la matrice génératrice G est définie telle que :

$$G = \left(\begin{array}{ccc|ccc} \overbrace{\quad\quad\quad}^k & & & \overbrace{\quad\quad\quad}^{n_1-k} & & \\ 1 & \cdots & 0 & b_{1,1} & \cdots & b_{1,n_1-k} \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & \cdots & 1 & b_{k,1} & \cdots & b_{k,n_1-k} \end{array} \right)$$

2.2 Code de Reed-Muller d'ordre 1

Après avoir appliqué le code de Reed-Solomon, on se retrouve avec un array d'éléments de $\mathbb{F}_{256}$ représentés par des octets. Le code de Reed-Muller visera à transformer chacun de ses octets par une autre matrice génératrice.

Les mots de code des codes Reed-Muller binaires consistent en des vecteurs d'évaluation de polynômes multivariés sur le corps binaire $\mathbb{F}_2$. La procédure d'encodage des codes RM associe les bits d'information stockés dans les coefficients polynomiaux au vecteur d'évaluation du polynôme. Considérons l'anneau de polynômes $\mathbb{F}_2[x_1, x_2, \dots, x_m]$ avec m variables. Pour un polynôme $f \in \mathbb{F}_2[x_1, x_2, \dots, x_m]$ et un vecteur binaire $z = (z_1, z_2, \dots, z_m) \in \mathbb{F}_2^m$, soit $\text{Eval}_z(f) := f(z_1, z_2, \dots, z_m)$ l'évaluation de f au vecteur z , et soit $\text{Eval}(f) := (\text{Eval}_z(f) : z \in \mathbb{F}_2^m)$ le vecteur d'évaluation de f dont les coordonnées sont les évaluations de f en tous les 2^m vecteurs dans $\mathbb{F}_2^m$. Les codes Reed-Muller avec paramètres m et r consistent en tous les vecteurs d'évaluation de polynômes avec m variables et de degré inférieur ou égal à r .

Soit un paramètre m un entier, alors le code de Reed-Muller, que l'on notera $RM(m, r)$ est l'ensemble :

$$RM(m, r) := \{\text{Eval}(f) : f \in \mathbb{F}_2[x_1, x_2, \dots, x_m], \deg(f) \leq r\}$$

Nous nous intéresserons ici au code de Reed-Muller d'ordre 1, avec $r = 1$. Nous pouvons définir sa matrice génératrice comme suit :

$$G = \begin{pmatrix} \text{Eval}(x_1) \\ \text{Eval}(x_2) \\ \vdots \\ \text{Eval}(x_m) \end{pmatrix}$$

La longueur de ce code sera $n = 2^m$ et sa dimension $k = m + 1$

3 Décodage de Reed-Solomon et Reed-Muller

Pour pouvoir déchiffrer les messages de HQC, on a besoin de décoder les messages transmis. On décrira dans cette section comment décoder des codes de Reed-Solomon et de Reed-Muller. Le principe de décodage est de trouver le mot de code le plus proche de ce qui a été transmis.

3.1 Décodage Reed-Muller

Supposons que l'on ait reçu un message $r = (x_1, \dots, x_n)$ d'un code $RM(m, 1)$ contenant n bits, calculons $F = ((-1)^{x_1}, \dots, (-1)^{x_n})$.

Ensuite on calcule $\hat{F} = H_{2^{m+1}} \times F$, en utilisant le produit classique de matrices, et avec $H_{2^{m+1}} = M_{m+1}^1 \times M_{m+1}^2 \times \dots \times M_{m+1}^{m+1}$

Pour calculer la matrice H_{27} , nous devons introduire ce qu'on appelle le produit de Kronecker entre deux matrices. Si $\mathbf{A} = [a_{ij}]$ est une matrice $m \times n$ et $\mathbf{B} = [b_{kl}]$ est une matrice $p \times q$, alors le produit de Kronecker $\mathbf{A} \otimes \mathbf{B}$ est la matrice par blocs $mp \times nq$:

$$\mathbf{A} \otimes \mathbf{B} = \begin{bmatrix} a_{11}\mathbf{B} & a_{12}\mathbf{B} & \cdots & a_{1n}\mathbf{B} \\ a_{21}\mathbf{B} & a_{22}\mathbf{B} & \cdots & a_{2n}\mathbf{B} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1}\mathbf{B} & a_{m2}\mathbf{B} & \cdots & a_{mn}\mathbf{B} \end{bmatrix} \quad (1)$$

Définissons la matrice M_m^i telle que :

$$M_m^i = I_{2^{m-i}} \otimes H_2 \otimes I_{2^{i-1}}$$

Le vecteur $\hat{F}$ contient en chaque position i la corrélation entre le mot transmis et l'encodage du mot i . Donc, une fois que l'on ait calculé $\hat{F}$, on cherche la position j , où $\|\hat{F}(j)\|$ est maximale. Le message que l'on cherchait à décoder est j si $\hat{F}(j) \geq 0$, et $2^m - j$ sinon.

3.2 Décodage Reed-Solomon

Maintenant intéresserons nous au décodage de Reed-Solomon. Supposons que l'on ait reçu un vecteur $v \in \mathbb{F}_{256}^n$. On introduit t comme $2t = n$. Voici les étapes à suivre pour décoder v .

1. **Calcul des syndromes** : Les syndromes $S_1, S_2, \dots, S_{2t}$ sont calculés à partir de v . Chaque syndrome S_i est une évaluation du polynôme d'erreur en un point du corps fini, ce qui permet de détecter la présence d'erreurs.
2. **Résolution du polynôme de déformation** : L'algorithme de Berlekamp-Massey est utilisé pour résoudre un système d'équations afin de trouver les coefficients du polynôme d'erreur, qui décrivent les positions et les valeurs des erreurs.
3. **Localisation des erreurs** : L'algorithme de Chien est appliqué pour localiser précisément les positions des erreurs en évaluant le polynôme d'erreur sur les points du corps fini.
4. **Correction des erreurs** : Une fois que les positions des erreurs sont identifiées, les valeurs des erreurs sont déterminées et corrigées en utilisant l'inverse de l'évaluation des syndromes.
5. **Récupération du message** : Après la correction des erreurs, le message original est récupéré en éliminant les erreurs et en retournant le mot code corrigé.